

```
import cv2 as cv
import numpy as np
from google.colab.patches import cv2_imshow
img = cv.imread('1.png', cv.IMREAD_GRAYSCALE)
assert img is not None, "file could not be read, check with os.path.exists()"
kernel = np.ones((5,5),np.uint8)
erosion = cv.erode(img,kernel,iterations = 1)
```

```
print ("Original Image: ")
cv2_imshow(img)
print ("Eroded Image: ")
cv2_imshow(erosion)
```

Original Image:



Eroded Image:



Dilation

```
dilation = cv.dilate(img,kernel,iterations = 1)
print ("Original Image: ")
cv2_imshow(img)
print ("Dilated Image: ")
cv2_imshow(dilation)
```

Original Image:



Dilated Image:



Opening

```
img_op_input = cv.imread('Original Image 2.png', cv.IMREAD_GRAYSCALE)
assert img is not None, "file could not be read, check with os.path.exists()"
opening = cv.morphologyEx(img_op_input, cv.MORPH_OPEN, kernel)

print ("Original Image: ")
cv2_imshow(img_op_input)
print ("Opened Image: ")
cv2_imshow(opening)
```

Original Image:



Opened Image:



Closing

```
img_cl_input = cv.imread('Original Image3.png', cv.IMREAD_GRAYSCALE)
assert img is not None, "file could not be read, check with os.path.exists()"
closing = cv.morphologyEx(img_cl_input, cv.MORPH_CLOSE, kernel)

print ("Original Image: ")
cv2_imshow(img_cl_input)
print ("Closed Image: ")
cv2_imshow(closing)
```

Original Image:



Closed Image:



Gradient

```
import cv2 as cv
from google.colab.patches import cv2_imshow
import numpy as np

img_grad_input = cv.imread('1.png', cv.IMREAD_GRAYSCALE)

# Check if image loaded properly
assert img_grad_input is not None, "File could not be read, check file path"

# Define kernel
kernel = np.ones((5,5), np.uint8)

# Apply Morphological Gradient
gradient = cv.morphologyEx(img_grad_input, cv.MORPH_GRADIENT, kernel)

# Display images
print("Original Image:")
cv2_imshow(img_grad_input)

print("Gradient Image:")
cv2_imshow(gradient)
```

Original Image:



Gradient Image:



Hit or Miss

```
import cv2 as cv
import numpy as np
from google.colab.patches import cv2_imshow

# Input image
input_image = np.array((
    [0, 0, 0, 0, 0, 0, 0, 0],
    [0, 255, 255, 255, 0, 0, 0, 255],
    [0, 255, 255, 255, 0, 0, 0, 0],
    [0, 255, 255, 255, 0, 255, 0, 0],
    [0, 0, 255, 0, 0, 0, 0, 0],
    [0, 0, 255, 0, 0, 255, 255, 0],
    [0, 255, 0, 255, 0, 0, 255, 0],
    [0, 255, 255, 255, 0, 0, 0, 0]
), dtype="uint8")

# Kernel value meaning:
# 1 = pixel must be foreground (white)
# -1 = pixel must be background (black)
# 0 = don't care (ignored)

kernel = np.array((
    [0, 1, 0],
    [1, -1, 1],
    [0, 1, 0]
), dtype="int")

output_image = cv.morphologyEx(
    input_image,      # input binaty image
    cv.MORPH_HITMISS, # morphological operatioon type
    kernel)           # pattern to detect

scale = 40 # evry pixel enlarge by 40 * 40
input_resized = cv.resize(input_image,
    (input_image.shape[1] * scale, input_image.shape[0] * scale), interpolation=cv.INTER_NEAREST)
output_resized = cv.resize(output_image,
    (output_image.shape[1] * scale, output_image.shape[0] * scale), interpolation=cv.INTER_NEAREST)

print("Input Image:")
cv2_imshow(input_resized)

print("Output Image (Hit or Miss):")
cv2_imshow(output_resized)
```

Input Image:



Output Image (Hit or Miss):

